

Fundamentos da Gestão Ágil

Disciplina: Gestão Ágil de Projetos de Software/Turma: 3º Semestre DSM.
Profª. Me. Waldinelly Costa

Dos Modelos Rígidos Tradicionais à Cultura de Adaptação Contínua

EIXO 01

Evolução dos Modelos

EIXO 02

Manifesto & Princípios

EIXO 03

Métodos: Scrum, Lean, Kanban, XP

EIXO 04

Valor, Iteração & Empirismo

Evolução dos Modelos de Gestão

1911



Taylorismo / Fordismo

Foco na previsibilidade mecânica, comando-e-controle estrito e tarefas hiper-especializadas.

Foco: Repetição fabril em massa.

1950s



Lean Manufacturing

Sistema Toyota: eliminação sistemática de desperdícios (*muda*) e fluxo puxado de valor.

Foco: Eficiência sem excessos.

1970s



Modelo Cascata

Fases lineares e sequenciais (Requisitos > Design > Código > Testes). Escopo fixo inicial.

Foco: Planejamento preditivo.

2001+



Movimento Ágil

Software empírico: ciclos curtos, feedback frequente, times autônomos e adaptação rápida.

Foco: Aprendizado e valor contínuo.

Cascata vs. Ágil: Uma Nova Mentalidade



Modelo Tradicional (Preditivo)

Entrega componentes isolados ao longo do projeto. O cliente não pode usar nada até a montagem final.



R
o
d
a



C
h
a
s
s
i



C
a
r
r
o
F
i
n
a
l

Risco: Se o mercado mudar no mês 6, todo o investimento anterior é perdido.

Resultado:

Feedback tardio e alto custo de mudança.



Modelo Ágil (Adaptativo)

Entrega um produto funcional e utilizável a cada ciclo (MVP), resolvendo a dor de transporte desde o dia 1.



S
k
a
t



B
i
c
i
l
e
t
a



C
a
r
r
o

Vantagem: O usuário experimenta, valida hipóteses e orienta a evolução do software.

Resultado:

Retorno antecipado e mitigação ativa de riscos.

O Manifesto Ágil (2001) e seu Mindset

A Ruptura com a Burocracia Instrumental

"O movimento ágil se diferencia das demais metodologias, ditas tradicionais, por conta do mindset criado e composto por ideias, valores e princípios que direcionam sua estrutura, seu funcionamento e sua manutenção. Diversos autores destacam os reportes de melhorias conquistadas por equipes que optaram pelas metodologias ágeis, descrevendo rapidez, qualidade e melhores processos de desenvolvimento."

(STELLMAN; GREENE, 2014; SCHWABER; SUTHERLAND, 2017.)

Em fevereiro de 2001, em Snowbird (Utah), 17 pioneiros formalizaram uma resposta definitiva aos processos engessados da engenharia de software tradicional.

Princípio Central: Mais do que aplicar cerimônias, o Ágil é uma cultura baseada em transparência, aprendizado e adaptação.



Os 4 Valores Fundamentais

1

Indivíduos e interações mais que processos e ferramentas

A comunicação direta entre desenvolvedores e clientes resolve bloqueios mais rápido que formulários burocráticos.

Prática DSM: Alinhamentos diários (Dailies) focados em remover impedimentos.

2

Software em funcionamento mais que documentação abrangente

O real indicador de sucesso é código testado rodando em produção gerando valor para o usuário final.

Prática DSM: Releases incrementais funcionando ao término de cada Sprint.

3

Colaboração com o cliente mais que negociação de contratos

Construção em parceria contínua com o Product Owner, refinando prioridades em vez de disputar escopos rígidos.

Prática DSM: Demonstrações frequentes com validação real do usuário.

4

Responder a mudanças mais que seguir um plano

Mudanças de mercado e novas ideias são bem-vindas e aproveitadas como oportunidade de vantagem competitiva.

Prática DSM: Repriorização estratégica do Backlog a cada novo ciclo.

Os 12 Princípios Ágeis

1. Foco no Cliente & Valor

- Satisfação do cliente através de entregas contínuas e antecipadas.
- Aceitar mudanças de requisitos mesmo em fases avançadas.
- Entregar software funcionando com frequência (semanas a meses).

Diretriz: Validar cedo hipóteses reais.

3. Pessoas & Comunicação

- Desenvolvedores e profissionais de negócio trabalham juntos diariamente.
- Construir projetos em torno de indivíduos motivados e confiáveis.
- Conversa face a face é o meio mais eficaz de transmitir informações.

Diretriz: Confiança e autonomia.

2. Excelência Técnica

- Software funcional é a principal medida de progresso.
- Atenção contínua à excelência técnica e bom design eleva a agilidade.
- Simplicidade: a arte de maximizar o trabalho não realizado.

Diretriz: Código limpo e sustentável.

4. Ritmo & Melhoria Contínua

- Desenvolvimento sustentável: manter um ritmo constante indefinidamente.
- As melhores arquiteturas e designs emergem de times auto-organizados.
- Em intervalos regulares, a equipe reflete e ajusta seu comportamento.

Diretriz: Retrospectivas regulares.

Métodos Ágeis: Scrum, Lean, Kanban e XP



Scrum

CICLOS & PAPÉIS

Framework iterativo em ciclos temporais fixos (**Sprints** de 1 a 4 semanas).

Papéis: Product Owner, Scrum Master e Desenvolvedores.

Eventos: Planning, Daily, Review e Retro.

Foco: Cadência e previsibilidade empírica.



Lean Software

FLUXO SEM DESPERDÍCIO

Filosofia focada em maximizar o fluxo de entrega de valor e eliminar o que não agrega (*muda*).

Decisões baseadas em evidências, aprendizado amplificado e entrega o mais rápido possível.

Foco: Eficiência e eliminação de excessos.



Kanban

FLUXO CONTÍNUO & WIP

Gestão visual do trabalho sem ciclos fixos pré-determinados.

Limita o Trabalho em Progresso (**WIP**), expõe gargalos e monitora métricas como *Lead Time*.

Foco: Visualização de fluxo e equilíbrio.



XP (Extreme)

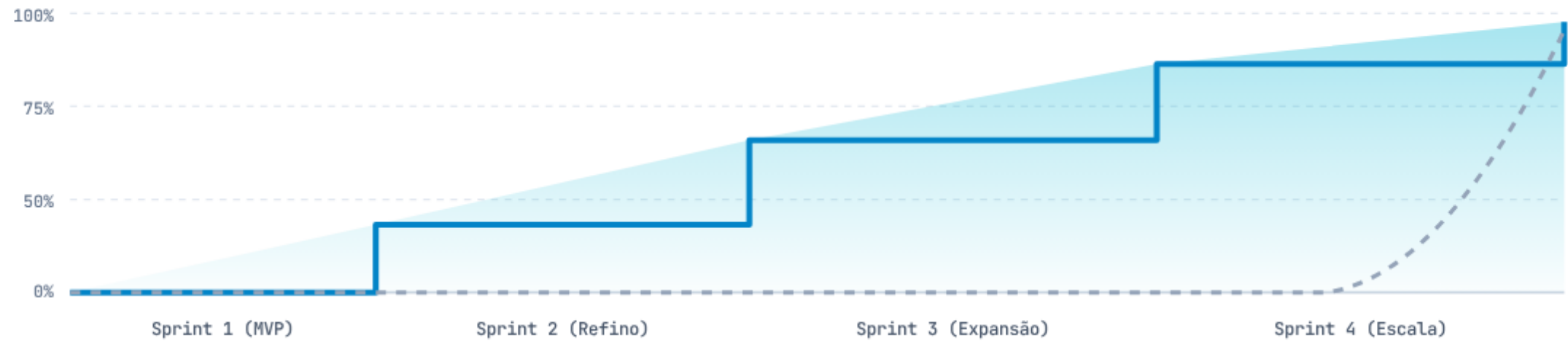
ENGENHARIA DE SOFTWARE

Conjunto de práticas de excelência técnica para código de alta qualidade.

Pilares: *Test-Driven Development (TDD)*, *Pair Programming*, *Integração Contínua* e *Refatoração*.

Foco: Qualidade intrínseca de código.

Conceitos de Valor e Retorno (ROI)



— **Gestão Ágil:** Geração precoce de valor (MVP), antecipação do ROI e aprendizado constante.

— **Modelo Cascata:** Retorno zero durante o ciclo; risco acumulado para a entrega final.

Operação: Iteração, Incremento & Empirismo

Iteração vs. Incremento

Duas forças complementares no ciclo de construção do software:

Iteração (Refinamento)

Aprimorar progressivamente uma solução existente através de ciclos contínuos de feedback.

Analogia: *Esboço > Cores básicas > Iluminação final da pintura.*

Incremento (Adição)

Adicionar módulos funcionais utilizáveis que expandem a capacidade do sistema.

Analogia: *Entrega do Login > Feed de Produtos > Checkout.*

Regra de Ouro: O software deve ser funcional ao fim de cada ciclo.

O Tripé do Controle Empírico

Decisões baseadas em fatos e experimentação real, não em premissas:

1. Transparência

O fluxo de trabalho, impedimentos e métricas devem ser visíveis para todo o time.

2. Inspeção

Avaliação frequente dos artefatos produzidos para detectar desvios (ex: *Sprint Review*).

3. Adaptação

Ajustar processos e prioridades no menor tempo possível (ex: *Sprint Retrospective*).

Conclusão: Corrigir a rota continuamente em vez de seguir um plano cego.

Referências Bibliográficas & Fechamento

STELLMAN, Andrew; GREENE, Jennifer. (2014)

Learning Agile: Understanding Scrum, XP, Lean, and Kanban. Sebastopol: O'Reilly Media.

SCHWABER, Ken; SUTHERLAND, Jeff. (2017/2020)

O Guia do Scrum: As Regras do Jogo. Scrum.org / Scrum Inc.

BECK, Kent et al. (2001)

Manifesto para o Desenvolvimento Ágil de Software. Disponível em: .

PRESSMAN, Roger S.; MAXIM, Bruce R. (2021)

Engenharia de Software: Uma Abordagem Profissional. 9. ed. Porto Alegre: McGraw Hill.

Síntese da Disciplina

A gestão ágil no Desenvolvimento de Software Multiplataforma não consiste em trabalhar sem direção, mas em **gerar valor sustentável**, aprender rapidamente por meio do empirismo e assegurar a qualidade do produto com excelência técnica.